

0x00 INTERACTIVE LEARNING STUDIO

Arrays & Linked Lists.

The two foundational ways to hold a sequence of data in memory — one contiguous and indexed, one scattered and pointer-linked. Master both, and you'll understand the trade-offs behind nearly every data structure built on top of them.



LEFT: array – contiguous, $\text{address} = \text{base} + (\text{index} \times \text{size})$ · RIGHT: linked list – scattered, each node stores the address of the next

~65 min

Estimated time

4

Modules

16

Quiz questions

None

Prerequisites

LEARNING OBJECTIVES

- Explain how arrays and linked lists actually live in memory, and why that shapes their performance.
- Implement and reason about core operations: access, search, insert, delete for both structures.
- Compare Big-O trade-offs and choose the right structure for a given real-world scenario.
- Build and manipulate singly, doubly, and circular linked lists — including reversal and cycle detection.

- Apply two-pointer, sliding-window, and fast/slow-pointer techniques to solve classic interview problems.
- Design a small system (e.g. an LRU cache or undo/redo stack) by combining both structures.

EXPECTED OUTCOMES

By the end, you'll be able to walk into a whiteboard interview and confidently implement a linked list from scratch, explain why `array[i]` is $O(1)$ but `list.get(i)` is $O(n)$, and pick the right structure when designing a system.



Reward System: Earn a badge for each module you complete with 75%+ on its quiz. Collect all 4 module badges plus the Final Challenge badge to unlock the **"Memory Architect"** completion certificate at the bottom of this page. Your progress bar at the top tracks it all live.

MODULE 1 / 4 · FOUNDATIONAL

FOUNDATIONS

Arrays: Contiguous Memory

Where the data lives, and why that single fact explains almost everything about how arrays behave.

What an array actually is

An array is a block of memory cells sitting **right next to each other**, all the same size, holding the same type of value. The array variable itself just remembers one thing: the **base address** — where the block starts.

Every other position is computed, not stored. To find element at index `i`, the computer does simple arithmetic:

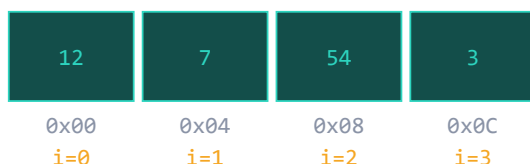
```
address(i) = base_address + (i × element_size)
```

```
// Example: base = 0x00, element_size = 4 bytes
// address(0) = 0x00 + (0 × 4) = 0x00
// address(2) = 0x00 + (2 × 4) = 0x08
```

Analogy — a street of identical houses: imagine houses numbered 100, 104, 108, 112 — evenly spaced, same size, on one street. If you know house 100 is the start and each house is 4 meters wide, you can walk directly to house #7 without visiting the ones before it. That's exactly how `array[7]` works: direct math, not a walk.

DIAGRAM: INDEX → ADDRESS

Array in memory (element size = 4 bytes)



`address(2) = 0x00 + (2×4)`
`= 0x08 → jumps straight there`

STATIC VS. DYNAMIC ARRAYS

A **static array** has a fixed size decided at creation (e.g. C's `int arr[10]`). A **dynamic array** (JavaScript arrays, Python lists, Java `ArrayList`) appears to grow freely, but under the hood it's still a fixed-size block — when it fills up, it silently allocates a new, bigger block (usually double the size), copies everything over, and frees the old one.

Common misconception: "Dynamic arrays can grow forever without cost." In reality, that resize-and-copy step is $O(n)$ when it happens — it's just rare enough (doubling means it happens $\sim \log(n)$ times) that the *average* cost per insertion stays $O(1)$. This is called **amortized $O(1)$** , not literally $O(1)$ every time.

CORE OPERATIONS & COMPLEXITY

OPERATION	TIME	WHY
Access by index	$O(1)$	Direct address math
Search (unsorted)	$O(n)$	Must check each element
Insert / delete at end	$O(1)^*$	*amortized; no shifting needed

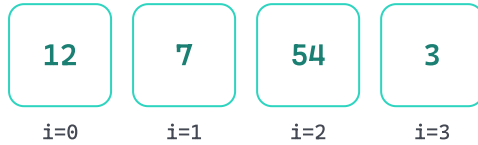
Insert / delete at start/middle

 $O(n)$

Every following element must shift

TRY IT: ARRAY SIMULATOR

Insert and delete values and watch how shifting works.



Reset to initial array.

 PRACTICAL EXERCISE

You have array [5, 8, 2, 9, 1]. Write out, step by step, what happens internally (which elements move, and where) when you insert 7 at index 2. Then state the final array and its Big-O cost.

in first step new empty value created then swap the last element to empty value and till it reach index 2 swap and when reach index 2 add 7

Expected reasoning: Elements at index 2,3,4 (2,9,1) each shift one position right → index becomes 3,4,5. Insert 7 at index 2. Final array: [5, 8, 7, 2, 9, 1]. Cost: $O(n)$ because up to n elements shifted, even though only one was "inserted."

Key Takeaways — Module 1

- Arrays store elements contiguously; index access is pure arithmetic → $O(1)$.
- Dynamic arrays resize by doubling and copying — amortized $O(1)$ insert at the end.
- Inserting/deleting anywhere but the end costs $O(n)$ due to shifting.
- Arrays are cache-friendly: contiguous memory means fast sequential reads.

Module 1 Quiz

Question 1 of 4

Why is array access by index $O(1)$?

Array access is direct math: $\text{base_address} + (\text{index} \times \text{element_size})$. No searching or walking required.

Question 2 of 4

A dynamic array in JavaScript just filled its internal capacity. What happens on the next `push()`?

Dynamic arrays resize (commonly doubling) and copy existing elements when capacity is exceeded — this single operation is $O(n)$, though it happens rarely enough that appends average out to $O(1)$ amortized.

Question 3 of 4

Why is inserting an element at the START of an array $O(n)$?

Inserting at the start requires shifting every other element one slot to the right, which touches up to n elements.

Question 4 of 4

Which property makes arrays especially fast for sequential reads on real hardware?

Because array elements sit next to each other in memory, the CPU can pull whole chunks into cache at once, making sequential access very fast — unlike scattered structures.

75%

3 of 4 correct



Great work — module badge earned! Next module unlocked.

MODULE 2 / 4 · FOUNDATIONAL

FOUNDATIONS

Linked Lists: Scattered, Connected Memory

No contiguous block, no index math — just nodes holding data and an address to "the next one."

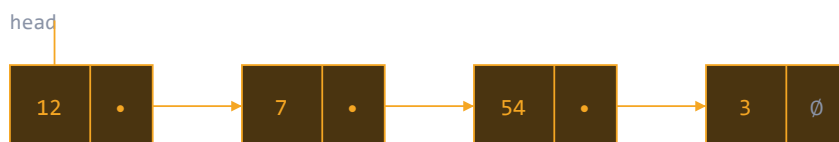
What a linked list actually is

A linked list is a chain of independent **nodes**. Each node holds two things: a value, and a pointer (memory address) to the next node. There's no shared block of memory — nodes can live anywhere, scattered across memory, connected only by these pointers.

```
struct Node {  
  value: any  
  next: Node | null  
}  
  
// head → [12|•]→[7|•]→[54|•]→[3|null]
```

Analogy — a treasure hunt: each clue doesn't tell you a location number; it tells you where to find the *next* clue. You can't skip to clue #4 — you must follow clue 1 → 2 → 3 → 4 in order. That's a linked list: no shortcuts, only "follow the pointer."

DIAGRAM: SINGLY LINKED LIST



SINGLY VS. DOUBLY VS. CIRCULAR

Singly linked list: each node points forward only. Simple, lower memory, but you can't walk backward.

Doubly linked list: each node has `next` AND `prev`. Enables backward traversal and $O(1)$ deletion when you already hold the node — at the cost of extra memory per node.

Circular linked list: the last node points back to the head instead of null. Useful for round-robin scheduling, playlists on loop, or turn-based games.

Common misconception: "Linked lists are always better for insertion because it's $O(1)$." Insertion is only $O(1)$ if you already have a reference to the node right before the insertion point. Finding that node in the first place is $O(n)$ — so inserting "at position 500" is still effectively $O(n)$ overall.

CORE OPERATIONS & COMPLEXITY

OPERATION	TIME	WHY
Access by index	$O(n)$	Must walk from head, no address math
Search	$O(n)$	Sequential traversal only
Insert/delete at head	$O(1)$	Just repoint the head pointer
Insert/delete at known node	$O(1)$	Just repoint neighboring pointers

ARRAYS VS. LINKED LISTS – SIDE BY SIDE

PROPERTY	ARRAY	LINKED LIST
Memory layout	Contiguous	Scattered
Random access	$O(1)$	$O(n)$
Insert at start	$O(n)$	$O(1)$
Extra memory per element	None	1–2 pointers
Cache performance	Excellent	Poor (pointer-chasing)

TRY IT: LINKED LIST SIMULATOR



Ready. Try: insert "9" at head.



PRACTICAL EXERCISE

You maintain a music queue as a singly linked list: head → SongA → SongB → SongC → null. A user wants to remove "SongB". Describe exactly which pointer(s) change, and why you need a reference to SongA (the node *before* SongB) to do it in $O(1)$.

Type your reasoning here ...

Key Takeaways — Module 2

- Linked lists trade random access for flexible, cheap insertion/deletion at known points.
- Doubly linked lists add backward traversal at the cost of extra memory.
- Circular lists remove the "end" — last node points back to head.
- $O(1)$ insert/delete claims only hold if you already have the relevant node reference.

Module 2 Quiz

Question 1 of 4

What does each node in a singly linked list store?

Singly linked list nodes hold a value plus a single "next" pointer. (Doubly linked lists add a "prev" pointer too.)

Question 2 of 4

Why is inserting at the HEAD of a linked list $O(1)$?

Inserting at the head just means creating a new node whose "next" points to the old head, then updating the head pointer — no other nodes move.

Question 3 of 4

What is the key structural difference between a circular and a singly linked list?

A circular linked list's tail node points back to the head, forming a loop — useful for round-robin or looping structures like playlists.

Question 4 of 4

Why does the claim "inserting into a linked list is always $O(1)$ " often mislead?

The insert itself is $O(1)$ only once you already hold a reference to the right node. Locating that node from the head is $O(n)$, so "insert at position 500" is $O(n)$ overall.

100%

4 of 4 correct



Great work — module badge earned! Next module unlocked.

MODULE 3 / 4 · APPLICATION

PRACTICAL APPLICATION

Applied Patterns & Techniques

The recurring tricks that show up across almost every array/linked-list interview problem.

Two-Pointer Technique (Arrays)

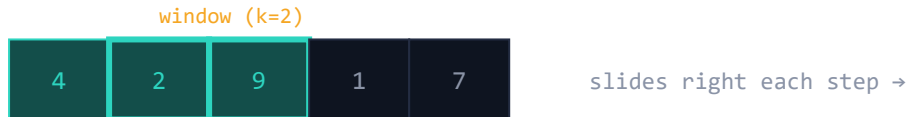
Instead of nested loops ($O(n^2)$), use two indices moving toward or alongside each other to solve problems in $O(n)$.

```
// Is array a palindrome? e.g. [1,2,3,2,1]
let left = 0, right = arr.length - 1;
while (left < right) {
  if (arr[left] !== arr[right]) return false;
  left++; right--;
}
return true;
```

Analogy: two people starting at opposite ends of a bookshelf, walking toward each other, checking that each pair of books matches — meeting in the middle instead of one person walking the whole shelf twice.

Sliding Window (Arrays)

Maintain a "window" (subarray) that expands and contracts as you scan once through the array — great for "max/min subarray of size k" or "longest substring" problems.



Fast & Slow Pointers (Linked Lists)

Two pointers traverse a linked list at different speeds (slow moves 1 step, fast moves 2). If the list has a cycle, they will eventually meet. If it doesn't, fast reaches `null` first.

```
function hasCycle(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}
```

Analogy — a race track: a runner twice as fast as another, on a circular track, will always eventually lap the slower one. On a straight track (no cycle), the fast runner just finishes first — they never "meet."

Reversing a Linked List

A classic mastery-check: reverse all the `next` pointers so the list points the other way, using three tracking pointers (`prev`, `current`, `next`).

```
function reverse(head) {  
  let prev = null, curr = head;  
  while (curr) {  
    let nextNode = curr.next;  
    curr.next = prev;  
    prev = curr;  
    curr = nextNode;  
  }  
  return prev; // new head  
}
```

Common misconception: "You can reverse a list by just relabeling head and tail." Nodes don't know they're "first" or "last" — only pointers define order. Every single `next` pointer must be individually flipped.

TRY IT: REVERSAL VISUALIZER



Original list: `12 → 7 → 54 → 3 → null`. Click "Step reversal" to walk through it.

PRACTICAL EXERCISE

Given array `[3, 1, 4, 1, 5, 9, 2, 6]`, use the sliding window technique (mentally) to find the maximum sum of any 3 consecutive elements. Show each window's sum as it slides.

Type your window sums here ...

Key Takeaways — Module 3

- Two-pointer and sliding-window patterns turn $O(n^2)$ array problems into $O(n)$.
- Fast/slow pointers detect cycles in linked lists without extra memory (Floyd's algorithm).
- Reversal requires tracking three pointers and flipping every link individually.
- These patterns are the building blocks of most array/linked-list interview questions.

Module 3 Quiz

Question 1 of 4

What problem does the two-pointer technique typically solve efficiently?

Two pointers moving toward or with each other let you avoid nested loops, turning many $O(n^2)$ problems into $O(n)$.

Question 2 of 4

In Floyd's fast/slow pointer cycle detection, what happens if the linked list has NO cycle?

Without a cycle, the fast pointer (moving 2 steps at a time) simply reaches the end (null) before ever meeting the slow pointer.

Question 3 of 4

When reversing a singly linked list iteratively, why do you need THREE tracking pointers (prev, curr, next)?

Before flipping `curr.next` to point backward (to `prev`), you must save a reference to the original next node — otherwise you lose access to the rest of the list.

Question 4 of 4

A sliding window of size k scans an array of size n . What is the typical time complexity when implemented efficiently (add/remove one element per slide)?

By adding the incoming element and subtracting the outgoing one (instead of re-summing the whole window), each slide is $O(1)$, giving $O(n)$ total.

75%

3 of 4 correct



Great work — module badge earned! Next module unlocked.

MODULE 4 / 4 · MASTERY

MASTERY

Design Decisions & Real Systems

Combining both structures, and knowing exactly when to reach for which one.

Choosing the right structure: a decision framework

Ask three questions:

1. **Do I need random access by index?** → Array wins ($O(1)$ vs $O(n)$).
2. **Do I insert/delete frequently at the front or middle?** → Linked list wins (no shifting).
3. **Do I care about memory locality / cache performance?** → Array wins (contiguous = fast CPU cache hits); linked lists scatter memory and cause "pointer chasing," which is slow on real hardware even though Big-O looks favorable.

Common misconception: "Big-O is the whole story." In practice, arrays often outperform linked lists even for operations where linked lists have better Big-O,

because CPU caches love contiguous memory. Big-O ignores constant factors and hardware reality — always sanity-check with real-world context.

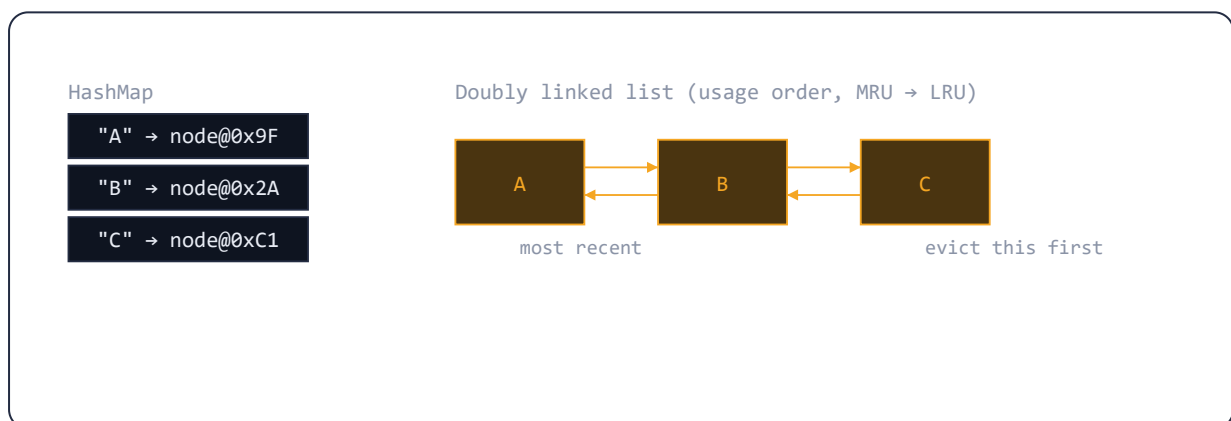
Case study: designing an LRU Cache

A Least-Recently-Used cache needs: $O(1)$ lookup by key, and $O(1)$ move-to-front / evict-oldest. Neither structure alone is enough:

- A **hash map** gives $O(1)$ lookup by key.
- A **doubly linked list** gives $O(1)$ reordering (move a node to the front) and $O(1)$ removal of the tail (oldest item) — because you already have direct node references from the hash map.

Combined: hash map stores key $\rightarrow$ node reference; doubly linked list maintains usage order. This is exactly how production LRU caches (e.g. in Redis-like systems) are built.

DIAGRAM: LRU CACHE STRUCTURE



Real-world uses at a glance

SYSTEM	STRUCTURE	WHY
Image pixel buffers	Array (2D)	Fixed size, constant random access needed
Browser back/forward history	Doubly linked list	Frequent insert/remove at ends, sequential navigation
Undo/redo stacks in editors	Linked list (as stack)	Push/pop at one end only, unpredictable size
Music "next up" queue	Circular linked list	Loops back to start automatically

Dynamic list rendering (UI)	Dynamic array	Iteration + occasional append is common; index access needed
--------------------------------	---------------	---

TRY IT: BUILD AN LRU CACHE (MINI)

Simulate a capacity-3 LRU cache. Access a key to move it to "most recently used."
When full, the least-recently-used key is evicted automatically.

Cache empty

Capacity: 3. Try accessing "A", "B", "C", then "A" again, then "D" to see eviction.



PRACTICAL EXERCISE

You're designing a text editor's "undo" feature. Would you use an array or a linked list for the undo stack? Justify your choice using the three-question framework above, and describe what operations you'd need.

Type your design reasoning here ...

Key Takeaways — Module 4

- Use a 3-question framework: random access? frequent front/middle edits? cache locality?
- Real systems often combine structures (e.g. hash map + doubly linked list for LRU cache).
- Big-O doesn't capture hardware reality — contiguous memory usually wins in practice.
- Matching structure to access pattern is the core skill of data structure design.

Module 4 Quiz

Question 1 of 4

Which combination of structures is classically used to build an $O(1)$ LRU cache?

The hash map gives $O(1)$ lookup by key (storing node references); the doubly linked list gives $O(1)$ reordering and $O(1)$ removal of the least-recently-used tail node.

Question 2 of 4

Why might an array outperform a linked list in practice even when Big-O suggests otherwise?

Big-O ignores constant factors and hardware behavior. Arrays' contiguous layout plays much better with CPU caches than a linked list's scattered nodes.

Question 3 of 4

For a browser's back/forward navigation history, which structure fits best and why?

Back/forward is inherently about moving one node at a time in either direction and occasionally cutting off "forward" history — a doubly linked list handles this naturally.

Question 4 of 4

According to the 3-question design framework in this module, when should you clearly prefer an array?

Arrays win when you need direct index access and want to benefit from contiguous-memory cache performance — the two classic array strengths.

100%

4 of 4 correct



Great work — module badge earned! Next module unlocked.

Final Practical Challenge

Bring together everything: memory layout, complexity trade-offs, and pointer manipulation.

Challenge: Detect and Remove a Cycle

You're given a singly linked list that *may* contain a cycle (a bug elsewhere in the codebase caused a node's `next` to accidentally point back to an earlier node). Your task, in plain-English pseudocode:

1. Detect whether a cycle exists (which technique from Module 3 applies?).
2. If a cycle exists, find the exact node where it begins.
3. Break the cycle by setting the appropriate node's `next` to `null`.

Write your full solution approach here ...

Cheat Sheet

ARRAY COMPLEXITY

Access	$O(1)$
Search	$O(n)$
Insert/delete end	$O(1)^*$
Insert/delete start/mid	$O(n)$

LINKED LIST COMPLEXITY

Access	$O(n)$
Search	$O(n)$
Insert/delete at head	$O(1)$
Insert/delete at known node	$O(1)$

KEY PATTERNS

Two-pointer	$O(n)$
Sliding window	$O(n)$
Fast/slow (cycle)	$O(n)/O(1)^{sp}$
Reversal	$O(n)$

PICK STRUCTURE WHEN ...

Need index access	Array
Frequent front insert	Linked list
Cache-sensitive	Array
Unknown/volatile size	Linked list

SUMMARY NOTES

Arrays are contiguous memory with $O(1)$ index math but $O(n)$ mid-insertions; linked lists are scattered nodes connected by pointers, giving $O(1)$ edits at known points but $O(n)$ access. Amortized doubling makes dynamic array appends cheap on average. Two-pointer, sliding-window, and fast/slow-pointer patterns solve most classic problems in $O(n)$. Real systems (like LRU caches) often combine both structures to get the best of each. Choosing between them is a trade-off exercise, not a "one is better" question — and real hardware cache behavior matters as much as Big-O.

KEEP GOING

Continue Learning

Where to go deeper after this tutorial.

BOOKS

- "Introduction to Algorithms" (CLRS) — Cormen, Leiserson, Rivest, Stein
- "Cracking the Coding Interview" — Gayle Laakmann McDowell
- "Grokking Algorithms" — Aditya Bhargava (visual, beginner-friendly)
- "The Algorithm Design Manual" — Steven Skiena

DOCUMENTATION

- MDN Web Docs — Array (JavaScript)
- Python docs — Data Structures (lists, collections.deque)
- Java docs — LinkedList & ArrayList class references
- cppreference.com — std::vector & std::list

RESEARCH PAPERS

- Floyd, R.W. (1967) — "Nondeterministic Algorithms" (cycle detection origins)
- Knuth, D. — "The Art of Computer Programming, Vol. 1: Fundamental Algorithms"
- Brent, R.P. (1980) — "An improved Monte Carlo factorization algorithm" (alt. cycle

COMMUNITIES

- r/learnprogramming, r/algorithms (Reddit)
- Stack Overflow — [data-structures] tag
- freeCodeCamp forums
- CS Discord servers (e.g. "The Programmer's Hangout")

detection)



PRACTICE PLATFORMS

- LeetCode — Arrays & Linked List problem sets
- HackerRank — Data Structures track
- NeetCode.io — curated roadmap with video explanations
- Codeforces — for timed practice under pressure



SEARCH KEYWORDS

amortized analysis

Floyd's cycle detection

doubly linked list

dynamic array resizing

LRU cache design

two pointer technique

sliding window algorithm

pointer chasing cache miss



ADDITIONAL AI PROMPTS FOR FURTHER LEARNING

"Quiz me on array vs linked list Big-O until I get 10 in a row correct without hesitation."

"Walk me through implementing a doubly linked list from scratch in [my preferred language], explaining each pointer update."

"Give me 5 LeetCode-style linked list problems ordered from easy to hard, with hints but not full solutions."

"Explain how a hash map + doubly linked list implements an LRU cache, then have me code it and review my solution."

Interactive Learning Studio · Arrays & Linked Lists · Built for deep, single-session
mastery